
作业二实验报告

王琳睿 241300009 (email: 1637928975@qq.com)

(南京大学 人工智能学院, 南京 210093)

1 任务一：介绍 Minimax 搜索

1.1 算法函数：

函数 `def minmax_decider` 通过一个 `maximizing_player` 开关参数将原本需要分开实现的 Max 和 Min 两个函数合并为单一函数。

需要传入的参数有：

`game,` 当前的游戏局势

`max_depth,` 还能够继续搜索的最大深度

`maximizing_player=True` 开关参数，决定当前需要选取最大值还是最小值

返回值为最好步骤的评估值和相应的走法。

1.2 实现逻辑：

MiniMax 算法的核心思想是假设博弈双方都会做出最优决策，通过递归探索可能的走法序列，为当前玩家选择最优策略。

首先对当前的游戏局势进行判断，如果游戏已经结束或者是达到了最大的深度限制，直接评估当前局面并返回，返回值为当前局势评估值和 `None`，不再继续搜索。

接着获取当前状态下所有有效走法，作为搜索的分支。

然后分别实现搜索中 AI 方和人类玩家方的选择逻辑。

AI 方（最大化方） 先初始化最大评估值为负无穷，遍历所有有效走法，为每个走法创建游戏状态副本，确保模拟走法不会影响原始游戏状态。再递归调用自身，改变开关参数切换为最小化方，比较递归调用返回的评估值，选择能产生最大评估值的走法；

而**人类方（最小化方）**相反，初始化最小评估值为正无穷，递归调用自身时切换为最大化方，选择能产生最小评估值的走法，其余逻辑与 AI 方相同。

最后返回为最好步骤的评估值和相应的走法。

1.3 时间空间复杂度：

都是 $O(b^d)$ ，其中 b 是平均分支因子，即平均有效走法数量， d 是搜索深度。

2 任务二：实现 Alpha-Beta 剪枝

2.1 算法函数：

与 MiniMax 算法基本相同，参数传入多加入：

alpha: α 值，表示当前路径上最大化玩家能保证的最低分数（默认负无穷）

beta: β 值，表示当前路径上最小化玩家能保证的最高分数（默认正无穷）

其余相同，返回值也相同。

2.2 实现逻辑：

Alpha-Beta 剪枝算法在 MiniMax 算法的基础上进行了优化，通过 $\alpha - \beta$ 剪枝来减少不必要的搜索节点，提高搜索效率。

与 MiniMax 的主要不同之处为新增加了剪枝的判断条件：

在最大化玩家回合，更新 α 值，如果当前节点的评估值 $\geq \beta$ ，则可以剪枝；

在最小化玩家回合，更新 β 值，如果当前节点的评估值 $\leq \alpha$ ，则可以剪枝。

其余与 MiniMax 算法相同。

2.3 速度变化：

分别对 Max_depth 为 4,6,7 时进行思考（搜索）时间的对比。使用同一种走棋方式。

1. Max_depth=4:

左为 MiniMax 算法，右为 Alpha-Beta 剪枝算法。

```
第1轮，AI思考时间：0.01秒
第2轮，AI思考时间：0.03秒
第3轮，AI思考时间：0.01秒
第4轮，AI思考时间：0.05秒
第5轮，AI思考时间：0.08秒
第6轮，AI思考时间：0.04秒
第7轮，AI思考时间：0.08秒
```

```
第1轮，AI思考时间：0.01秒
第2轮，AI思考时间：0.01秒
第3轮，AI思考时间：0.01秒
第4轮，AI思考时间：0.02秒
第5轮，AI思考时间：0.03秒
第6轮，AI思考时间：0.02秒
第7轮，AI思考时间：0.04秒
```

平均思考时长:MiniMax 算法 0.043 秒，Alpha-Beta 剪枝算法 0.02 秒。

Alpha-Beta 剪枝算法比 MiniMax 算法用时缩短约 53.4%

2. Max_depth=6:

左为 MiniMax 算法，右为 Alpha-Beta 剪枝算法。

第1轮，AI思考时间：3.31秒
第2轮，AI思考时间：8.85秒
第3轮，AI思考时间：12.10秒
第4轮，AI思考时间：17.60秒
第5轮，AI思考时间：14.19秒
第6轮，AI思考时间：49.29秒
第7轮，AI思考时间：26.24秒

第1轮，AI思考时间：0.55秒
第2轮，AI思考时间：0.91秒
第3轮，AI思考时间：0.84秒
第4轮，AI思考时间：1.52秒
第5轮，AI思考时间：0.85秒
第6轮，AI思考时间：1.22秒
第7轮，AI思考时间：0.58秒

平均思考时长:MiniMax 算法 18.80 秒，Alpha-Beta 剪枝算法 0.92 秒。

Alpha-Beta 剪枝算法比 MiniMax 算法用时缩短约 95.1%

3. Max_depth=7:

左为 MiniMax 算法，右为 Alpha-Beta 剪枝算法。

（本来定的是最大深度为 8，因 MiniMax 思考时间过长，会出现“未响应”情况影响时间判断，因此改成最大深度为 7 并且只选取前 4 轮）

第1轮，AI思考时间：14.23秒
第2轮，AI思考时间：42.13秒
第3轮，AI思考时间：156.07秒
第4轮，AI思考时间：395.75秒

第1轮，AI思考时间：0.77秒
第2轮，AI思考时间：1.74秒
第3轮，AI思考时间：3.84秒
第4轮，AI思考时间：11.08秒

平均思考时长:MiniMax 算法 152.045 秒，Alpha-Beta 剪枝算法 4.36 秒。

Alpha-Beta 剪枝算法比 MiniMax 算法用时缩短约 97.1%

总结：算法走的最大深度越大，博弈树的规模呈指数增长，每一轮思考时间也呈指数级增长，符合 MiniMax 算法的时间复杂度预期；

同时，随着最大深度提升，Alpha-Beta 剪枝算法展现出巨大的效率优势，速度提升愈发明显：深度越大，被剪掉的子树规模就越大，因此节省的计算量就越高。

在有限的时间或资源下，Alpha-Beta 剪枝算法的 AI 可以思考的更深远，从而做出更具有前瞻性的决策。

3 任务三：改进 evaluate_game_state 函数

3.1 原功能介绍：

函数 `def evaluate_game_state(game)` 用于对当前游戏局面进行评分，帮助 AI 判断局面的好坏。

包括函数 `def calculate_stability(game)` (`def neighbors(row, col)` `def is_stable_disk(row, col).)`

首先设置不同评估因素的权重系数，区分各因素的重要性。

接着分别计算各因素：

棋子数量差异，权重 1.0、

行动力差异（当前玩家和对手的合法移动数量之差），权重 2.0、

边缘占据情况（不包含角落），权重 2.5，

稳定性（calculate_stability 中计算，在 calculate_stability 部分介绍），权重 3.0，

角落占据情况，权重 5.0。

最后综合所有因素，按权重计算最终评估值。

其中函数 calculate_stability:

首先定义内部函数 neighbors，用于获取指定位置的所有相邻位置（8 个方向）。

接着，定义棋盘的不同区域，角落，边缘，内部区域，并初始化稳定棋子计数器为 0。

然后定义内部函数 is_stable_disk：如果一个棋子的所有相邻位置都是同色棋子，或位于边缘或角落，则认为稳定。

最后遍历所有区域，统计满足条件的稳定棋子数量，返回稳定的棋子的总数。

3.2 改进之处：

3.2.1 细化棋盘位置划分及加入位置权重

在定义棋盘的内部时，使用 `inner_region = [(i, j) for i in range(2, 6) for j in range(2, 6)]`，range(2, 6) 为 {2, 3, 4, 5}，这只能包含 (2, 2) 到 (5, 5) 的 4*4 区域，缺少其外围一圈的位置。这导致遍历棋盘时会遗漏。

我原本以为应该改成：`inner_region = [(i, j) for i in range(1, 7) for j in range(1, 7)]`，但是查询网上的黑白棋教程^[1]之后，我发现棋盘的分区比代码中实现的更加复杂：（图来源于网络）

	a	b	c	d	e	f	g	h
1		C	A	B	B	A	C	
2	C	X					X	C
3	A							A
4	B							B
5	B							B
6	A							A
7	C	X					X	C
8		C	A	B	B	A	C	

“如果你把棋子下在 C 点和 X 点，那么对方很可能占领角落。”^[1]

因此原代码中给棋盘分区的逻辑基本正确（需要修改边的包含区域），意图应该是遍历棋盘中位置优势较大的区域来寻找稳定的棋子。

但同时，我了解到棋盘各位置落子都有好坏之分，不仅仅是**计算稳定性**时需要用到划分，这也是一个**重要的评估因素**：

可以直接定义棋盘各

位置的权重：

`[-25, -45, 1, 1, 1, 1, -45, -25],`

`[10, 1, 3, 2, 2, 3, 1, 10],`

`[5, 1, 2, 1, 1, 2, 1, 5],`

```
[5, 1, 2, 1, 1, 2, 1, 5],
[10, 1, 3, 2, 2, 3, 1, 10],
[-25, -45, 1, 1, 1, 1, -45, -25],
[500, -25, 10, 5, 5, 10, -25, 500]][2]
```

（网上不同教程资料中各位置权重数值不同，但是相对大小关系与以上基本一致。）

注：应用到原代码中时应与原权重系数统一数量级！

并直接将 `corner_occupancy` 和 `edge_occupancy` 合并为一个因素 `positional_score`（棋子位置加权因数）。

将棋子位置加权因数加入评估，可以更加全面地评估整个棋盘，使得 AI 倾向于占领权重高的位置。

3.2.2 阶段化权重动态调整

在棋局的不同阶段，各评估因素的重要性有所变化。

通过查阅资料和询问 AI（deepseek）我得到以下参数：

```
def get_dynamic_weights(game): 1 个用法 吕 Arike
    """根据游戏阶段动态调整权重"""
    total_disks = sum(1 for row in game.board for cell in row if cell != 0)
    game_phase = total_disks / 64 # 0-1, 0=开局, 1=终局

    if game_phase < 0.3: # 开局阶段 (0-19子)
        return {
            'coin_parity': 0.1, # 棋子数量: 不重要
            'mobility': 0.5, # 行动力: 最重要
            'stability': 0.2, # 稳定性: 次要
            'positional_score': 0.2 # 位置分数: 重要
        }
    elif game_phase < 0.7: # 中局阶段 (20-44子)
        return {
            'coin_parity': 0.25, # 棋子数量: 重要性提升
            'mobility': 0.3, # 行动力: 仍然重要
            'stability': 0.3, # 稳定性: 重要性提升
            'positional_score': 0.15 # 位置分数: 重要性降低
        }
    else: # 终局阶段 (45-64子)
        return {
            'coin_parity': 0.5, # 棋子数量: 最重要
            'mobility': 0.1, # 行动力: 不重要
            'stability': 0.25, # 稳定性: 重要
            'positional_score': 0.15 # 位置分数: 辅助作用
        }
```

那么自然能想到棋盘各位置权重也可以随着游戏的各个阶段而动态变化：

```
if game_phase < 0.3: # 开局阶段
    return [
        # 开局：强调角落，避免 C 位，控制边界
        [0.6, -0.25, 0.15, 0.08, 0.08, 0.15, -0.25, 0.6],
        [-0.25, -0.3, -0.08, -0.08, -0.08, -0.08, -0.3, -0.25],
        [0.15, -0.08, 0.05, 0.02, 0.02, 0.05, -0.08, 0.15],
        [0.08, -0.08, 0.02, 0.01, 0.01, 0.02, -0.08, 0.08],
        [0.08, -0.08, 0.02, 0.01, 0.01, 0.02, -0.08, 0.08],
        [0.15, -0.08, 0.05, 0.02, 0.02, 0.05, -0.08, 0.15],
        [-0.25, -0.3, -0.08, -0.08, -0.08, -0.08, -0.3, -0.25],
        [0.6, -0.25, 0.15, 0.08, 0.08, 0.15, -0.25, 0.6]
    ]
elif game_phase < 0.7: # 中局阶段
    return [
        # 中局：平衡发展，内部位置价值提升
        [0.5, -0.15, 0.2, 0.12, 0.12, 0.2, -0.15, 0.5],
        [-0.15, -0.2, 0.03, 0.02, 0.02, 0.03, -0.2, -0.15],
        [0.2, 0.03, 0.1, 0.06, 0.06, 0.1, 0.03, 0.2],
        [0.12, 0.02, 0.06, 0.04, 0.04, 0.06, 0.02, 0.12],
        [0.12, 0.02, 0.06, 0.04, 0.04, 0.06, 0.02, 0.12],
        [0.2, 0.03, 0.1, 0.06, 0.06, 0.1, 0.03, 0.2], # 5
        [-0.15, -0.2, 0.03, 0.02, 0.02, 0.03, -0.2, -0.15],
        [0.5, -0.15, 0.2, 0.12, 0.12, 0.2, -0.15, 0.5]
    ]
else: # 终局阶段
    return [
        # 终局：所有位置都有正价值，强调棋子数量
        [0.4, 0.08, 0.25, 0.15, 0.15, 0.25, 0.08, 0.4],
        [0.08, 0.05, 0.12, 0.09, 0.09, 0.12, 0.05, 0.08],
        [0.25, 0.12, 0.18, 0.14, 0.14, 0.18, 0.12, 0.25],
        [0.15, 0.09, 0.14, 0.11, 0.11, 0.14, 0.09, 0.15],
        [0.15, 0.09, 0.14, 0.11, 0.11, 0.14, 0.09, 0.15],
        [0.25, 0.12, 0.18, 0.14, 0.14, 0.18, 0.12, 0.25],
        [0.08, 0.05, 0.12, 0.09, 0.09, 0.12, 0.05, 0.08],
        [0.4, 0.08, 0.25, 0.15, 0.15, 0.25, 0.08, 0.4]
    ]
```

3.2.3 稳定棋子判断依据修改

原函数的稳定棋子判断条件是所有相邻格子都是己方棋子或位于边缘或角落。

关于稳定棋子，我搜索网络后得到：

“当一个棋子的四个方向均不可翻转时，这个棋子为稳定子。而一个方向上不可翻转有 2 种可能：1、该方向上一边接边界或己方的稳定子 2. 该方向上两边接对方稳定子”^[3]

但根据前两次修改的经验，我意识到适合的稳定性判断的条件也可能随着游戏的进行而不同。

（为简化问题，我选择只将游戏过程分为两个部分：）

前半部分，应保守估计，原判断条件比较严格，即可适用。

后半部分，参考查到的资料，将判断条件改为：先找出四个角上是否有稳定棋子，接着对每个棋子每个方向都进行上述两个条件的检查，这里稍微放宽标准，若满足条件的方向（共八个方向，四个正交方向，四个对角线方向）不少于 4 个，则认为其满足稳定棋子的条件。

3.2.4 对手行动力计算方法修改

原函数的逻辑是创建了一个全新的游戏对象，而不是当前棋盘状态的副本，会导致获取完全错误的信息。

应该将

```
opponent valid moves = len(
    OthelloGame(player mode=-game.current player).get valid moves()
)
```

修改为（使用 copy.deepcopy）

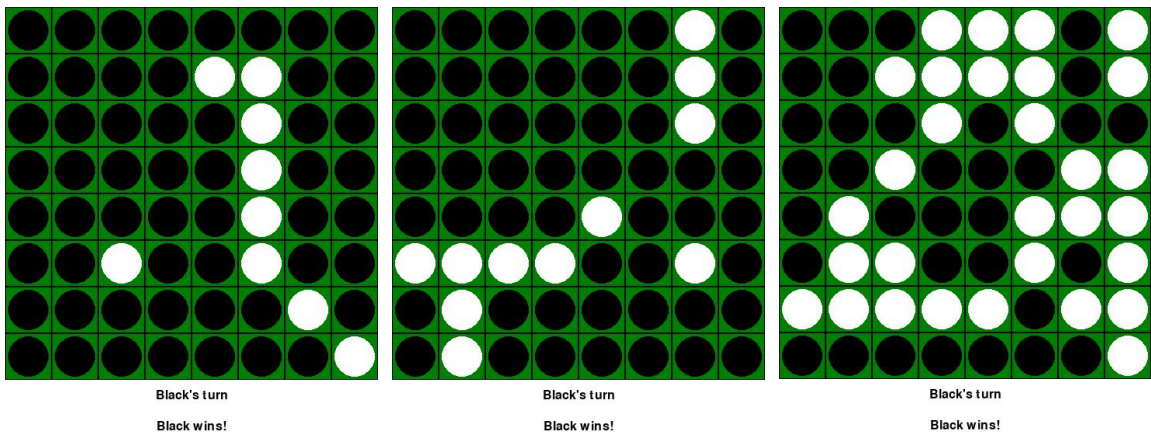
```
temp_game = copy.deepcopy(game)
temp_game.current player = -game.current player
opponent valid moves = len(temp_game.get valid moves())
```

（或者使用 minmax_decider 函数里的创建方法）

3.3 改进结果1：

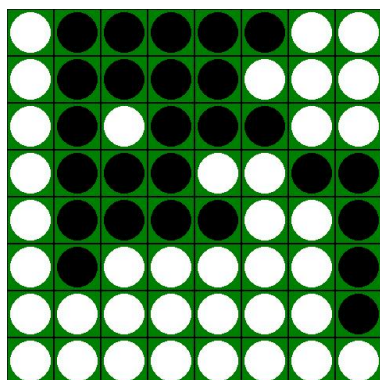
让双方 AI，一个使用改进前的算法，一个使用改进后的算法，分别在最大搜索深度为 4, 6, 7 时对战。

（1）改进前为白棋，改进后为黑棋（先走），从左到右深度为 4, 6, 7



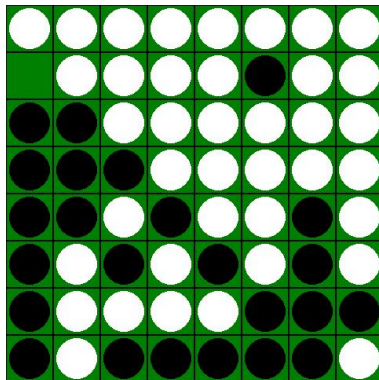
结果是改进后的算法均胜利，且优势相对较大。

(2) 改进前为黑棋，改进后为白棋（后走），从左到右深度为 4, 6, 7



Black's turn

White wins!



White's turn

White wins!

Max_depth=7 时黑棋胜利

改进后的算法胜利两局。优势没有先走时明显。

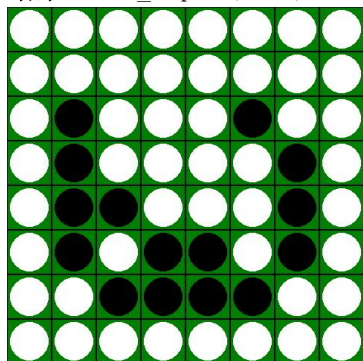
并且当搜索深度增大时，劣势将被放大。

我认为这种情况可能是当前的权重设置更符合先手的走棋思路（攻击而非防守），应根据先后手的情况区分权重分配。

3.4 再次修改及结果2:

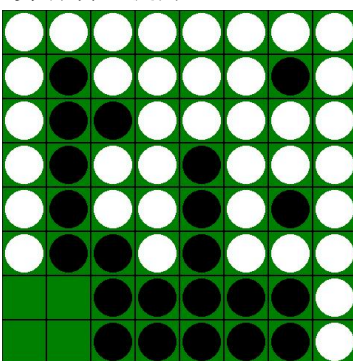
修改权重设置，相对于先手稍微降低棋子数量权重，提升行动力权重。

结果: Max_depth 在 4 和 6 时优势有明显提升:



Black's turn

Invalid White/Black gain.



Black's turn

White wins!

（深度为 7 时仍无法成功，但是劣势相对更小）

因此我认为搜索深度也是一个需要优化的点，不必强求算法在各深度均表现良好，实际应用时应注意避免搜索深度过深。例如，控制在 6 之内。

3.5 reference:

- [1] [【入门】强大的角和危险的 X 点 - 知乎](https://zhuanlan.zhihu.com/p/32320751) (https://zhuanlan.zhihu.com/p/32320751)
- [2] [黑白棋 AI: 局面评估+AlphaBeta 剪枝预搜索 - 知乎](https://zhuanlan.zhihu.com/p/35121997) (https://zhuanlan.zhihu.com/p/35121997)
- [3] [如何通过编程判断棋盘上的棋子稳定性-CSDN 博客](https://blog.csdn.net/u012501320/article/details/31830009)

(https://blog.csdn.net/u012501320/article/details/31830009)

4 任务四：介绍 MTD(f)算法(函数 mtd_f)与 minimax 算法的异同

4.1 算法函数：

MTD(f)算法 `def mtd_f(game, guess, max_depth)` 是一种基于零窗口搜索的优化算法，它通过动态调整评估函数的猜测值(guess)，并由 guess 决定初始化的 alpha 与 beta，多次调用 Alpha-Beta 搜索来找到最优解。

需要传入的参数有：

`game`, 当前的游戏局势；

`guess`, 对当前局面真实评估值的初始估计；

`max_depth`, 还能够继续搜索的最大深度；

返回值：最好步骤的评估值和相应的走法。

4.2 算法实现：

首先初始化边界：建立搜索区间，设置下界为负无穷，上界为正无穷，真实的最优值一定在这个区间内。使用传入的 `guess` 参数作为搜索的起始点。

接着当下界小于上界时（意味着还没有找到精确的最优值，需在区间内继续搜索）进行迭代搜索，每次迭代：

1. 如果 `g` 等于 `lower_bound`，将 `beta` 设置为 `g + 1`，避免陷入死循环；
（死循环情况：如果返回 `g ≥ beta`，则 `lower_bound` 更新为 `g`，边界没有变化，陷入死循环）
否则，`beta` 设置为 `g`；
2. 设置零窗口，设置 `alpha = beta - 1`, `beta = beta`，将窗口的宽度压缩至 1，并使用 alpha-beta 剪枝算法函数进行搜索下一步行动以及最优得分。
3. 根据返回的 `g` 值的结果更新 `lower_bound` 和 `upper_bound`。
4. 当 `lower_bound == upper_bound`，即收敛到同一个值时循环结束。
此时 `g` 就是真实的最优值，`best_move` 就是最佳走法。

4.3 与minimax算法的异同：

1.相同点：

理论基础相同：都基于 MiniMax 理论框架，假设对手最优响应

目标一致：目标都是找到当前局势下的最优走法

结果等价性：保证找到与标准 MiniMax 相同的最优解

递归结构：都采用递归方式遍历游戏树

评估函数依赖：都依赖相同的局面评估函数

2.不同点：

MTD 使用零窗口迭代搜索；minimax 算法使用全窗口单次搜索。零窗口能够触发更多剪枝，提高搜索效率。

MTD 多次调用 Alpha-Beta；minimax 算法仅进行单次完整搜索。

MTD 需要初始估值猜测；minimax 算法不需要初始猜测。MTD 对初始猜测值的依赖较强，在配合好的初始猜测值时，性能提升更加明显。在迭代深化中，使用前一层结果作为下一层 `guess`

效果最佳^(*)。

4.4 *说明 (reference) :

“在迭代深化中，使用前一层结果作为下一层 `guess` 效果最佳^(*)”。这句是我在询问 AI 算法细节时看见的，原函数中并没有实现这一方法，是基础构建块，本身只负责单次深度的搜索，没有包含迭代深化的逻辑。可以构建外层的迭代深化函数，对不同的最大搜索深度循环调用 `MTD` 算法函数来实现这一优化策略。